

PuckBot: Predictive Interception and Striking Control for IIWA Air Hockey

Ryan Quinn
CSAIL
MIT
Cambridge, MA
rquinn1@mit.edu

Guillermo Mendoza
CSAIL
MIT
Cambridge, MA
gdmen@mit.edu

Abstract—High-speed manipulation remains a formidable challenge in robotics. This paper introduces PuckBot, a simulation framework comprising two KUKA IIWA robotic arms engaged in competitive air hockey. We propose a control architecture that integrates 2D kinematic prediction with offensive-defensive strategies. The system estimates puck trajectories, accounting for friction and wall deflections, to calculate optimal end-effector interception points. This work demonstrates the effectiveness of trajectory planning in high-velocity scenarios and provides a reproducible simulation environment for further research. Our evaluation consists of three core environments: a single PuckBot that is only playing defense, a single PuckBot that is only playing offense, and two PuckBots on each end simulating a realistic game of air hockey. The results demonstrate robust defensive capabilities, achieving a mean intersection rate of 82% and a save percentage of 73% in single-agent trials. In head-to-head tournament play, the system maintained consistent puck control with over 55 touches per match, though offensive conversion rates remained lower due to unoptimized strike dynamics. These findings confirm that predictive kinematic planning can achieve reliable high-speed interception without the need for reinforcement learning.

I. INTRODUCTION (RYAN)

A core challenge in dynamic robotic manipulation is executing precise interceptions of rapidly moving objects under significant time constraints. This form of manipulation requires real-time estimation of position and velocity to plan well-positioned intercepts quickly. The slightest of miscalculations can result in incorrect predictions and mistimed movements, ultimately producing undesirable results. In these high-speed scenarios, reactive control strategies are insufficient because system latency exceeds available reaction time. Therefore, to achieve strong performance, a system must implement predictive frameworks capable of accurately estimating future object states over short time horizons.

In this paper, we present a simulation framework where two robotic IIWA arms simulate a game of air hockey [Fig. 1]. Each arm is equipped with *trajectory planning* logic that allows it to predict the motion of the puck and run inverse kinematics to intercept accordingly. Our environment consists of an air hockey table, a hockey puck, and two mallets, all which exhibit standard physics. We maximize offensive and defensive performance, allowing the robots to mimic human gameplay.

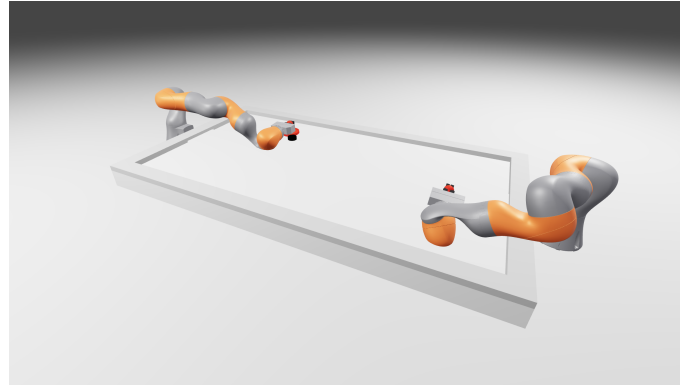


Fig. 1. The simulation consists of two KUKA IIWA robots on opposite sides of the air hockey table. Each IIWA grasps and manipulates the end of their mallet.

Air hockey is extremely fast-paced and requires precise manipulation. This type of environment is where most robots struggle. Players have milliseconds to decide where to move their paddle so that it intersects the puck in an advantageous way. For example, a shot that is on target (or has a velocity towards the opposing player's goal) provides an advantage because it requires the opposing player to act defensively. Even small miscalculations can lead to own goals, awkward bounces, or easy opportunities for opponents.

This makes air hockey a compelling benchmark for robotics. It simulates the fast manipulation of objects and requires real-time estimation of the puck's position and velocity. These core challenges are fundamental to robotics and extend to broader applications in movement (walking/running), sports, and everyday tasks. Producing desirable strikes is difficult and requires the player to adjust the speed, orientation, and timing of their movements simultaneously.

For reference, air hockey is a popular tabletop game where two players use circular paddles to strike a sliding puck into the opponent's goal. The puck glides with near-zero friction, bounces off walls, and must be hit both quickly and accurately. Success depends on both offense (striking the puck in a way that pressures the opponent) or defense (positioning the paddle to block or redirect incoming shots). Players earn points by scoring goals, with the winner typically being the first to reach

seven.

By integrating motion planning, strategy, and inverse kinematics, we will design a simulation that is more lightweight and just as effective as current models which use both direct manipulation and reinforcement learning. To showcase its effectiveness, we simulate our design against previous ones in a head-to-head game of air hockey.

II. RELATED WORK (GUILLERMO)

Robot air hockey has emerged as a valuable benchmark for studying high-speed, contact-rich manipulation. Early systems largely relied on classical control pipelines combining vision-based state estimation, analytic trajectory prediction, and rule-based strategy selection. These approaches are computationally efficient and highly interpretable, enabling robots to react reliably under well-modeled conditions [1].

More recent work has pushed these classical ideas toward greater speed and reactivity. Liu et al. demonstrate that deterministic pipelines can still achieve high-performance play by refining state estimation and optimizing intercept strategies [1]. Their results show that well-designed analytic systems can outperform learning-based methods when timing constraints are tight and model uncertainty is low. Similar challenges arise in other high-speed tasks such as robot table tennis and juggling, where the robot must constantly adjust to unpredictable object motion. Work in these areas highlights how trajectory prediction and precise timing are essential for consistent performance [4], [6].

The Robot Air Hockey Challenge has encouraged research into the development of reinforcement learning, imitation learning, and hybrid architectures that attempt to learn broader strategies while retaining classical control for low-level execution [2]. While these methods can discover more nuanced offensive behavior, they often struggle with precise timing and reliability [2].

Our approach builds on these insights by combining classical predictive modeling with simple strategy logic. Our architecture consists of 2D kinematic prediction, fast intercept selection, and real-time trajectory generation for the IIWA arm. Unlike the previously discussed systems focusing solely on defense or specialized striking, our method aims to balance both blocking incoming shots while calculating an offensive path.

III. METHODOLOGY (RYAN AND GUILLERMO)

PuckBot consists of 3 main modules, with the system architecture shown in Fig. 2:

- 1) **Puck Trajectory Prediction:** We simulate the puck's 2D motion by stepping forward from the current state in small time increments dt [Fig. 3]. At each step, we record the position, apply friction by scaling the velocity, and move the position forward. When the next position crosses a y-boundary, it reflects the position and inverts the y-velocity by the restitution coefficient. If the puck crosses an x-boundary, we assume a goal and stop the program early.

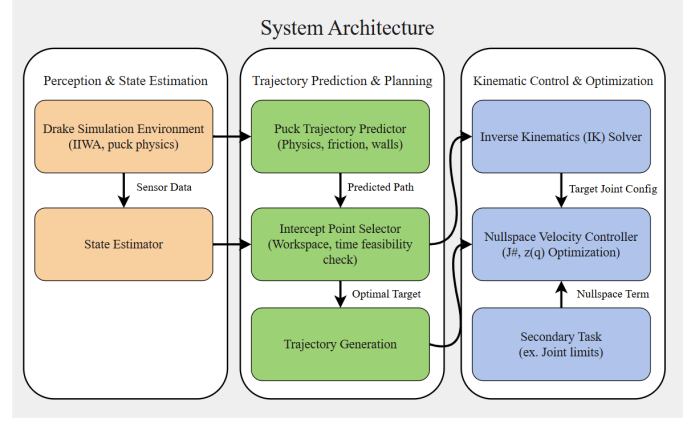


Fig. 2. The core architecture behind PuckBot consists of physical states/environments, trajectory planning and prediction, and kinematics control for the robot. We pass information from our environment (puck state, robot state, etc.) to our trajectory planning functionality to ensure that predictions are accurate and efficient. The trajectory planning information is then passed to our kinematics systems, which determine the logistics behind how our robot will complete an interception successfully.

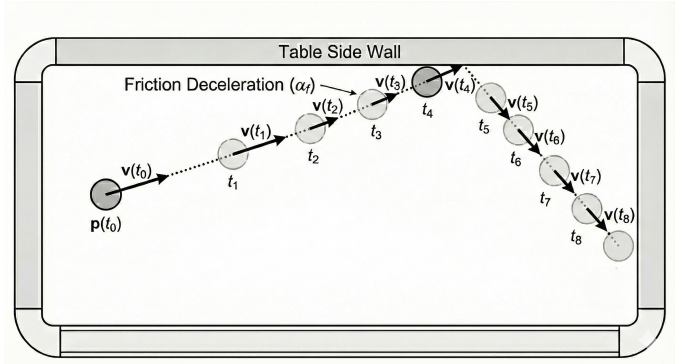


Fig. 3. Our trajectory planning system uses information from the puck's past motion (previous location and velocity) to predict where it will be after a certain amount of time.

- 2) **Intercept Planning:** By predicting the motion of the puck, we can derive a set of *feasible robot candidate positions* that will successfully intercept the puck. A feasible position is confined to the configuration of the robot's workspace. We generate a *linearly interpolated* trajectory between the current and target joint configurations that intercepts the puck desirably.
- 3) **Kinematics:** For each feasible robot candidate position, we efficiently solve inverse kinematics to find joint angles and check whether the robot can reach them within the available time given joint velocity limits.

A. Environment Setup (Guillermo)

We use the Drake robotics framework as the primary simulation environment for our robotic experiments. The simulation scene is configured to replicate a standard competitive air hockey setup, with the Air Hockey Table, sourced from the established Air Hockey Challenge benchmark, placed at the global origin of the simulation. Two 7 Degrees of Freedom

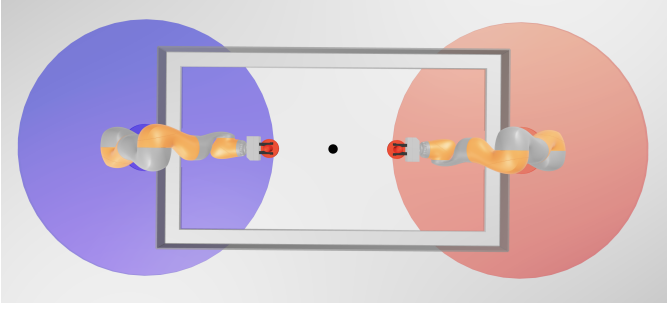


Fig. 4. The colored circles represents the workspace boundaries for each IIWA robot. Both robots cannot reach the puck outside of their designated section.

(DOF) KUKA IIWA robotic arms are mounted at opposite ends of the table to serve as the players. The mallets are grasped initially by the IIWA robots via their end-effectors. The Puck Dynamics are modeled to exhibit realistic physics crucial for the game: the puck's motion across the table surface includes a moderate, velocity-dependent friction force, causing its speed to decay linearly over time, simulating the low, near-zero friction characteristic of air hockey. Collisions between the puck and table boundaries are modeled as perfectly elastic, producing a lossless nonzero impulse based interaction that reverse the tangential component of the puck's velocity while preserving its speed (i.e. $v_f = -v_i$). In contrast, puck-paddle interactions are modeled as fully dynamic contacts, where the puck generates an escape velocity proportional to the amount of force received by the paddle. Collision with the end boundaries (goals) results in a goal being scored and the puck's state being reset to a predefined starting position.

B. Puck Dynamics and Prediction (Ryan and Guillermo)

The puck trajectory prediction system employs a two-dimensional kinematic model that operates in the horizontal plane of the air hockey table, assuming the puck maintains a constant height z corresponding to the table surface. This simplification reduces complexity while maintaining sufficient accuracy for intercept planning, as the primary motion occurs in the xy -plane with little to no vertical variation. The state representation projects the three-dimensional puck state and velocity:

$$\mathbf{p}(t) = [x, y, z]^T \quad (1)$$

$$\mathbf{v}(t) = [v_x, v_y, v_z]^T \quad (2)$$

onto their two-dimensional counterparts:

$$\mathbf{p}(t) = [x, y]^T \quad (3)$$

$$\mathbf{v}(t) = [v_x, v_y]^T \quad (4)$$

The core dynamics assume constant-velocity motion between discrete time steps, with modification occurring only through friction forces and wall collisions. During each integration step of duration Δt , the position is updated according to the standard kinematic equation:

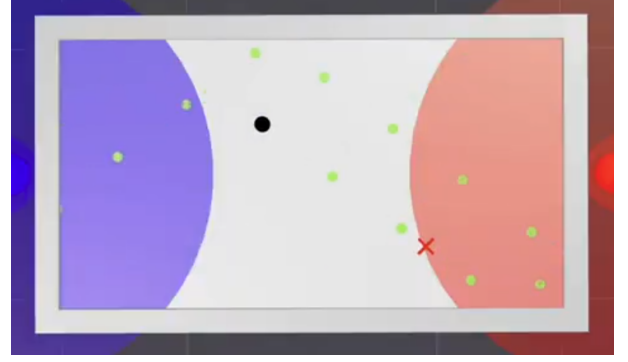


Fig. 5. The puck's predicted trajectory is outlined by the light green circles. The red X demonstrates the location where PuckBot aims to intercept the puck.

$$\mathbf{p}_{2d}(t + \Delta t) = \mathbf{p}_{2d}(t) + \mathbf{v}_{2d}(t) \cdot \Delta t \quad (5)$$

This approach represents the *explicit Euler integration* scheme, which trades numerical precision for efficiency and performance. We default the time step Δt to 0.01 seconds to balance prediction accuracy and compute.

Friction is modeled as a velocity-dependent deceleration force that acts opposite to the direction of motion. The implementation utilizes a simple friction model where the velocity magnitude decreases *linearly* over time. At each time step, the velocity vector is scaled by a friction factor, resulting in the updated velocity:

$$\alpha_f = \max\left(0, 1 - \frac{a_f \cdot \Delta t}{\|\mathbf{v}_{2d}\|}\right) \quad (6)$$

$$\mathbf{v}_{2d}(t + \Delta t) = \alpha_f \cdot \mathbf{v}_{2d}(t) \quad (7)$$

The maximum operation with zero prevents the velocity from reversing direction due to overshoot in a single time step. The default friction deceleration parameter $a_f = 0.1m/s$ models moderate table friction, which can be tuned during experimentation.

C. Intercept Planning (Ryan)

The intercept planning system coordinates robot motion to intercept a moving puck at a predicted future position and time [Fig. 5]. This requires physical reachability, kinematic feasibility, and overall robot coordination. We transform a continuous puck trajectory into a sequence of *candidate interception points*, followed by evaluating each candidate through inverse kinematics and velocity constraints.

Our goal is to plan the robot's motion so that it intercepts the puck in minimal time. Our system generates a joint-space trajectory for the first feasible solution encountered, which prioritizes computational efficiency and responsiveness. This enables quicker reactions to puck motion.

The planning process starts by querying the puck trajectory predictor to obtain a discrete-time sequence of predicted puck positions over a specified time horizon, $T_{horizon}$, typically 2.0 seconds. The predictor returns pairs of times and positions:

$$\{(t_i, \mathbf{p}_i)\}_{i=1}^N$$

where each $\mathbf{p}_i \in \mathbb{R}^3$ is the predicted 3D puck position at time t_i . N depends on the horizon duration and the prediction time step, which is defaulted to 0.02 seconds. This process creates a rich sampling of potential intercept opportunities along the puck's path.

The first filtering stage removes predicted positions outside the robot's workspace boundaries [Fig. 4]. This is meant to eliminate infeasible candidates early, reducing computational burden for inverse kinematics solving.

For each sampled candidate, the system first computes the available time window:

$$\Delta t_{\text{available}} = t_{\text{intercept}} - t_{\text{current}} \quad (8)$$

where t_{current} is the current simulation time. Candidates with $\Delta t_{\text{available}} < 0.1$ seconds are discarded, since there is not enough time for our robot to react. The target intercept position is adjusted to account for the paddle height above the table surface.

Even when an IK solution exists, the robot may not be able to execute the required motion within the available time window due to joint velocity limits. The time feasibility check evaluates whether the robot can traverse from $\mathbf{q}_{\text{current}}$ to $\mathbf{q}_{\text{target}}$ within $\Delta t_{\text{available}}$ without exceeding any joint velocity limits.

Once a candidate passes both IK solving and time feasibility checking, the system generates a joint-space trajectory connecting the current and target configurations. The trajectory uses *linear interpolation* to create a sequence of waypoints forming a straight-line path between the initial and final joint configurations.

D. Kinematics (Guillermo)

The configuration of the KUKA IIWA is defined by its seven joint angles, $\mathbf{q} \in \mathbb{R}^7$, within a bounded joint space:

$$\mathbf{q} = \{q_i \mid i \in \{1, \dots, 7\}\}, \quad q_i^{\min} \leq q_i \leq q_i^{\max} \quad (9)$$

Our system utilizes Task-Space Velocity Control to command the end-effectors desired motion, $\mathbf{V}_d^G$, which is a spatial velocity vector (linear and angular velocity). The relationship between the joint velocities, $\dot{\mathbf{q}}$, and the end-effector velocity, $\mathbf{V}^G$, is defined by the Jacobian matrix, $\mathbf{J}(\mathbf{q})$:

$$\mathbf{V}^G = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (10)$$

To find the required joint velocities, $\dot{\mathbf{q}}$, that achieve the desired task-space velocity, $\mathbf{V}_d^G$, we employ the Moore-Penrose pseudo-inverse of the Jacobian, $\mathbf{J}^\#$, to obtain the minimum-norm solution:

$$\dot{\mathbf{q}} = \mathbf{J}^\# \mathbf{V}_d^G \quad (11)$$

The 7 Degrees of Freedom (DOF) of the IIWA arm exceed the 6 DOF required to define the end-effector's pose

(position and orientation) in 3D space, resulting in kinematic redundancy. This allows us to achieve the desired end-effector motion while simultaneously optimizing a secondary objective by leveraging the Jacobian's Nullspace.

We augment the joint velocity command with a Nullspace motion term, $\mathbf{z}(\mathbf{q})$, which performs self-motion without affecting the end-effector's pose:

$$\dot{\mathbf{q}} = \mathbf{J}^\# \mathbf{V}_d^G + (\mathbf{I} - \mathbf{J}^\# \mathbf{J}) \mathbf{z}(\mathbf{q}). \quad (12)$$

The term $\mathbf{P} = (\mathbf{I} - \mathbf{J}^\# \mathbf{J})$ is the Nullspace projection matrix. This matrix projects any vector $\mathbf{z}(\mathbf{q})$ onto the null space of the Jacobian.

The secondary task, represented by $\mathbf{z}(\mathbf{q})$, is specifically used in PuckBot for singularity and joint-limit avoidance. By defining $\mathbf{z}(\mathbf{q})$ as the gradient of a self-motion potential function (e.g., $H(\mathbf{q})$), we ensure that the robot's configuration remains robustly controllable and far from mechanical limits while executing the primary intercept motion. For instance, $\mathbf{z}(\mathbf{q})$ can be set as:

$$\mathbf{z}(\mathbf{q}) = k \cdot \nabla H(\mathbf{q}) \quad (13)$$

where k is a scaling factor and $H(\mathbf{q})$ penalizes proximity to joint limits or singularities, thereby improving numerical robustness and overall stability during high-speed maneuvers.

IV. EVALUATION (RYAN AND GUILLERMO)

In this section, we present the experimental results from running our air hockey simulation. We evaluate our algorithm under three core environments:

- 1) **Defense** - The environment renders a single IIWA robot and a random position and velocity for a puck, which is initialized to move towards its own goal. Our program's defensive capabilities are measured by the number of intercepted hits and the save percentage (shots on goal saved divided by total shots on goal).
- 2) **Offense** - The environment renders a single IIWA robot and a stationary puck at a random position within its reach. In this environment, our program's offensive capabilities are measured by the percentage of shots on target.
- 3) **Tournament** - The environment renders two IIWA robots that compete against each other in a head-to-head game mode. The puck, upon reset, begins with a randomly selected velocity and direction towards the robot who just forfeited a goal. This environment tracks the metrics from both the offense and defense modes for both robots, as well as the total puck touches.

A. Evaluation Metrics

The following metrics were used to quantify the performance of the robot in the environments described above:

- 1) **Intersection Rate**: Percentage of successful intersections between the IIWA paddle and the puck (regardless of goal trajectory).

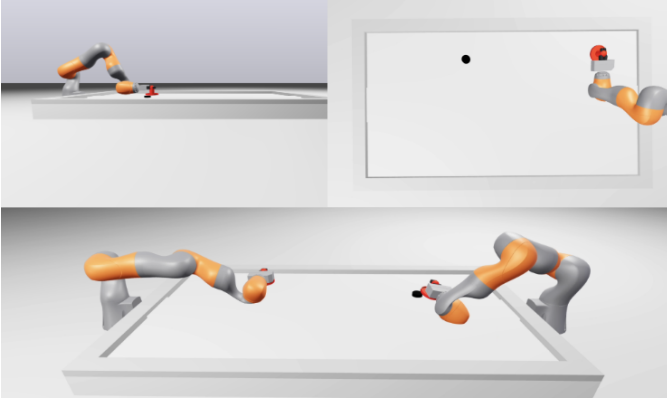


Fig. 6. Visual overview of the three experimental environments evaluated in the Drake simulation. **(Top Left)** The **Offensive** setup, where a single IIWA robot attempts to strike a stationary puck into the goal. **(Top Right)** The **Defensive** setup, where the robot must intercept a puck initialized with a random velocity vector towards the goal. **(Bottom)** The **Tournament** mode, featuring two competing agents in a continuous head-to-head match.

- 2) **On Target Goal Rate:** Percentage of success in converting a hit to a goal, calculated using the predicted trajectory of the puck.
- 3) **Save Percentage:** Shots on goal saved divided by the total number of shots on goal.
- 4) **Total Puck Touches (Tournament Mode):** The number of total touches within the match to characterize possession.

B. Attacking Performance

We evaluated the offensive capabilities of the single-arm setup over 100 simulations, each lasting 30 seconds. Table I details the results for intersection success and goal conversion.

TABLE I
OFFENSIVE (SINGLE ARM) MEASURED METRICS ($N = 100$, $t = 30s$)

Metric	Mean	Std Dev	Min	Max
Intersection Rate	0.83	0.15	0.73	0.91
On Target Goal Rate	0.13	0.063	0.021	0.15

C. Defense Performance

The defensive performance was similarly evaluated over 100 simulations of 30 seconds each. Table II presents the statistics for intersection rate and save percentage.

TABLE II
DEFENSE (SINGLE ARM) MEASURED METRICS ($N = 100$, $t = 30s$)

Metric	Mean	Std Dev	Min	Max
Intersection Rate	0.82	0.09	0.75	0.94
Save Percentage	0.73	0.12	0.61	0.84

D. Tournament Mode Performance

Finally, we evaluated the robots in a competitive head-to-head scenario. Table III breaks down the performance by arm.

TABLE III
TOURNAMENT PERFORMANCE METRICS BY ROBOT ARM ($N = 100$, $t = 30s$)

Robot	Metric	Mean	Std Dev	Min	Max
Left Arm	Intersection Rate	0.73	0.13	0.61	0.85
	Save Percentage	0.64	0.15	0.42	0.82
	On Target Goal Rate	0.09	0.012	0.024	0.17
	Total Puck Touches	55.21	10.24	32	64
Right Arm	Intersection Rate	0.79	0.05	0.65	0.94
	Save Percentage	0.74	0.13	0.62	0.91
	On Target Goal Rate	0.13	0.023	0.021	0.15
	Total Puck Touches	58.45	9.15	39	71

E. Discussion and Analysis

The experimental results highlight a significant divergence between the system’s ability to intercept the puck versus its ability to accurately aim it. While the intersection rate remained high across all environments, the offensive conversion rate was notably lower.

We hypothesize that this discrepancy stems from our intersection algorithm. While the solver accurately calculates *where* to meet the puck, it does not currently optimize for the puck’s returning velocity vector upon collision. In a real-world setting, human players often curve their strike trajectory to cancel out specific velocity components, effectively redirecting the puck’s momentum toward the goal. Our current approach acts more like a linear wall than a dynamic striker, resulting in successful contacts that lack precise directional intent.

Qualitatively, the robot exhibited a distinct bias toward a specific elbow configuration, often holding the paddle to the side for the duration of the match. The robot maintains this biased pose and sweeps across the width of the table to reach the intersection point. Surprisingly, this motion produced some of the highest return velocities in gameplay, suggesting that this elbow-out stance provided greater kinematic leverage than a neutral pose.

Another notable emergent behavior was the utilization of walls and corners. Although we did not explicitly program defensive zones, the trajectory prediction algorithm incorporates wall collision mechanics. Consequently, the model occasionally opted to remain stationary near a wall, correctly calculating that the puck’s natural bounce would result in an interception without requiring active robot movement.

Finally, we observed a slight performance asymmetry in the Tournament mode, where the Right Arm consistently outperformed the Left Arm. Given identical policy code, this is likely due to minor initialization biases in the physics engine. Regardless of the current offensive metrics, we are satisfied with these initial results, as they demonstrate robust defensive capabilities and complex emergent behaviors that were not explicitly hard-coded.

V. CONCLUSION (RYAN)

In this work, we introduced PuckBot, a predictive control framework for robotic air hockey that addresses the fundamental challenges of high-speed manipulation. By integrating trajectory estimation and kinematics, we demonstrated that

a predictive architecture allows a robot to intercept fast-moving objects effectively. Our system utilizes 2D kinematic prediction to account for friction and wall reflections, paired with velocity control and nullspace optimization to navigate workspace limitations.

Experimental evaluations show that PuckBot maintains high intercept accuracy (>80%) and defensive reliability in high-velocity scenarios. Furthermore, the use of Drake allowed for a reproducible environment that models contact-rich physics. Future work will explore the integration of reinforcement learning to develop strategic decision-making, allowing the robot to not only intercept the puck but to learn the nuances of deceptive human gameplay. Ultimately, the methodologies presented in PuckBot provide an excellent foundation for applying predictive control and trajectory planning to broader classes of dynamic object manipulation.

REFERENCES

- [1] P. Liu, D. Tateo, H. Bou-Ammar and J. Peters, "Efficient and Reactive Planning for High Speed Robot Air Hockey," 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 2021, pp. 586-593, doi: 10.1109/IROS51168.2021.9636263.
- [2] P. Liu et al., "A Retrospective on the Robot Air Hockey Challenge: Benchmarking Robust, Reliable, and Safe Learning Techniques for Real-world Robotics," arXiv.org, 2024. <https://arxiv.org/abs/2411.0571>
- [3] R. L. Bishop and M. W. Spong, "Control of a robotic air hockey system," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Leuven, Belgium, May 1998, pp. 289-294.
- [4] M. Bühler, D. E. Koditschek, and P. J. Kindlmann, "A family of robot juggling tasks," Int. J. Robot. Res., vol. 13, no. 2, pp. 101-118, Apr. 1994.
- [5] M. Diehl, H. J. Ferreau, and N. Haverbeke, "Efficient numerical methods for nonlinear MPC and moving horizon estimation," in Nonlinear Model Predictive Control, L. Magni, D. M. Raimondo, and F. Allgöwer, Eds. Berlin, Heidelberg: Springer, 2009, pp. 391-417.
- [6] K. Mülling, J. Kober, and J. Peters, "Learning to select and generalize movements in robot table tennis," IEEE Trans. Robot., vol. 29, no. 4, pp. 934-946, Aug. 2013.
- [7] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, Robotics: Modelling, Planning and Control. London, UK: Springer, 2009.